COST AND PERFORMANCE · DOMAIN 6 - COST & PERFORMANCE OPTIMIZATION

Compute choices and what they cost

Section 6. The four principles, and the levers that actually move the bill.

SECTION 6 of 10 WEIGHT 13% TOPIC Compute cost

admin@sthithapragna.com

Compute type is the largest single lever on the bill, and it is chosen before anybody writes a line of code.

admin@sthithapragna.com

Use job compute for anything non-interactive

- The docs are plain: non-interactive workloads **cost significantly less** on job compute than on all-purpose.
- A production pipeline left running on an interactive cluster is the single most common avoidable overspend.
- Each job can run on new compute, which also **isolates workloads** from one another.
- Or a **multitask job can reuse compute** across its tasks, so start-up is paid once per job rather than per task.

> ON THE EXAM
>
> Notice the second option. Reusing compute across tasks is how you avoid paying start-up time repeatedly in a job with many small steps.

SQL warehouses, and Photon by default

- For interactive SQL, a SQL warehouse is described as **the most cost-efficient engine**.
- **All SQL warehouses come with Photon by default.** It is not something you enable.
- Serverless warehouses add **intelligent workload management** for handling many concurrent queries.
- Running interactive SQL on an all-purpose cluster pays a higher rate for a less suitable engine.

> ON THE EXAM
>
> Photon being on by default for warehouses is worth remembering, because on classic compute it is a choice you have to make.

When serverless wins on cost despite a higher rate

- Non-serverless warehouses start in **minutes**, so people leave them running rather than wait.
- Serverless starts and scales **in seconds**, so instant availability and idle termination are both achievable.
- Serverless also **scales down earlier** than non-serverless.
- So for **bursty** workloads the higher rate is offset by not paying for idle time at all.

> ON THE EXAM
>
> A higher per-unit rate can still be cheaper. What matters is rate multiplied by time, and serverless changes the time.

The rules of thumb the docs give

FAMILY	USE IT FOR
Memory optimised	Machine learning, and heavy shuffle and spill workloads.
Compute optimised	Structured Streaming, and maintenance jobs such as optimize and vacuum.
Storage optimised	Workloads that benefit from caching, such as ad-hoc and interactive analysis.
GPU optimised	Specific machine learning and deep learning workloads only.
General purpose	In the absence of specific requirements.

ON THE EXAM

Note the second row. Streaming and maintenance jobs want compute-optimised, which is not what most people reach for by default.

Three numbers that matter more than worker count

- **Total executor cores** determines the maximum parallelism available.
- **Total executor memory** determines how much can be held before spilling to disk.
- **Executor local storage** is used for spills during shuffles, and for caching.
- Databricks runs **one executor per worker node**, so executor and worker mean the same thing here.

> ON THE EXAM
>
> People size clusters by worker count. The docs point at cores, memory and local disk instead, which is what a workload actually consumes.

Autoscaling, and the streaming exception again

- Autoscaling adds workers for the demanding phases of a job and removes them afterwards, beating a static size.
- **It has limitations scaling down for Structured Streaming workloads.**
- Databricks recommends **Lakeflow pipelines with enhanced autoscaling** for streaming instead.
- That is the same recommendation module 7 gave from the pipelines side, arriving here as a cost argument.

> ON THE EXAM
>
> Module 6 said do not autoscale a hand-written stream. This is why, and this is the alternative.

Auto-termination and pools

- Configure **auto-termination on all interactive compute**. Idle clusters are pure waste.
- If start-up times are the reason people leave clusters running, use **pools** of idle ready-to-use instances.
- **Databricks does not charge DBUs while instances sit idle in a pool.**
- The instance provider does still bill for them, so a pool is not free. It moves the cost, and shortens start-up.

> ON THE EXAM
>
> Both halves of that pool fact matter. No DBUs, but the cloud provider still charges for the machines.

What a compute policy can enforce

- Require **autoscaling** with a set minimum number of workers.
- Require **auto-termination** with a sensible value, the docs suggest an hour as an example.
- Restrict which **VM instance types** may be chosen, and set maximum worker counts.
- Apply a **spot instance strategy**, and define standard sizes so choices are consistent.

> ON THE EXAM
>
> A policy fails cluster creation at submission. That is a much better control than a monthly cost review noticing it afterwards.

Judge the total, not the rate

- Photon accelerates SQL and DataFrame calls, and is compatible with Spark APIs. **No code changes.**
- It bills at a higher rate, and **a faster job can still be cheaper overall** because billing is by time.
- The docs say regular jobs **should be evaluated** to see whether they are not only faster but also cheaper.
- So it is a measurement, not an assumption. It does not help every workload equally.

> ON THE EXAM
>
> The same reasoning as serverless. Rate multiplied by time is what you pay, and a higher rate over less time can win.

Tagging, and the thing you cannot undo

- Tag workspaces, clusters, warehouses and pools. Tags propagate to **usage logs and to cloud provider resources**.
- Recommended minimum: **business unit** and **project**, plus **environment** where dev and production must be separated.
- **Tags only affect future usage. Missing tags cannot be added to past events.**
- So start detailed. You can ignore a tag later; you cannot backfill one.

> ON THE EXAM
>
> This is the only genuinely irreversible decision in the module. Everything else can be changed next week.

What a DBU does and does not include

- For classic compute, total cost is **DBUs plus the virtual machine, disk and network** charges.
- **For serverless, the DBU cost already includes the virtual machine cost.**
- So comparing a serverless DBU rate against a classic one is not comparing like with like.
- A migration to serverless can look like DBUs rising while infrastructure falls, with total spend barely moving.

> ON THE EXAM
>
> This single difference explains most confused conversations about whether serverless is more expensive.

Budgets, usage policies and system tables

- **Budgets** track account-wide or filtered spending, with email notification when a threshold is reached.
- **Usage policies** attribute serverless usage by tagging activity incurred by an assigned user.
- The **billable usage system table** is where custom tags land, so attribution becomes a query.
- Billable usage can also be downloaded as a file through the account API for offline analysis.

> ON THE EXAM
>
> Serverless needs usage policies specifically, because there is no cluster of yours to hang a tag on.

Sharing does not replicate, and may still cost

- Sharing avoids replication, which is the whole argument from module 15.
- But your cloud vendor **may charge data egress fees** when you share across clouds or regions.
- That cost lands on the **provider**, not the recipient, and there is guidance on monitoring it.
- So the zero-copy argument is about storage and staleness, not about network charges.

> ON THE EXAM
>
> A cost that appears on the cloud bill rather than the Databricks one, which is exactly why it goes unnoticed.

Always-on streaming is a choice, not a requirement

- Streaming suggests always on, and always on means paying for compute **every hour of the day**.
- If the business needs fresh data **every few hours**, a few runs a day meets that requirement.
- Databricks recommends the **availableNow trigger** for incremental workloads without low latency requirements.
- Same code, same checkpoint, dramatically less compute. Module 11's point, seen as a cost decision.

> ON THE EXAM
>
> Ask what the business actually requires, not what the source looks like. A continuous source does not compel continuous compute.

Cheaper capacity, with one rule

- Spot instances use excess cloud capacity at a lower price, and Databricks supports clusters built on them.
- **The first instance, the Spark driver, should always be on-demand.**
- They suit workloads where it is acceptable to take longer because instances were evicted.
- **Fleet instance types** let Databricks pick the physical instance with the best price and availability.

> ON THE EXAM
>
> Losing a worker to eviction costs you some rework. Losing the driver ends the job. That asymmetry is the whole rule.

What to carry into the exam

- **Job compute for non-interactive work.** The largest single lever, and the most common mistake.
- Serverless and Photon can be **cheaper at a higher rate**, because you pay rate multiplied by time.
- Autoscaling **does not scale down well for streaming**. Use pipelines with enhanced autoscaling.
- **Serverless DBU rates already include the VM cost.** Classic rates do not.
- **Tags cannot be backfilled.** Spot is fine for workers, never for the driver.

> ON THE EXAM
>
> That completes Section 6 at 13%. Section 7, security and compliance, is next at 10%.

> A finance review finds that 40% of Databricks spend comes from a cluster that runs continuously. Investigation shows it hosts eleven scheduled production pipelines, all configured to attach to it, plus occasional notebook work by two engineers.
>
> The cluster has autoscaling enabled and is correctly sized.

What is the largest single saving available?

A. Move the eleven pipelines to job compute, which costs significantly less for non-interactive work.

B. Reduce the autoscaling maximum to cap the cluster's size.

C. Enable Photon on the cluster to shorten pipeline runtimes.

D. Add spot instances to the worker pool to lower the instance cost.

◆ PAUSE AND DECIDE

What is the largest single saving available?

A Move the eleven pipelines to job compute, which costs significantly less for non-interactive work.

Scheduled pipelines are non-interactive. The docs are explicit that they cost significantly less on job compute.

B Reduce the autoscaling maximum to cap the cluster's size.

The scenario says it is correctly sized. Capping it constrains work without addressing the rate.

C Enable Photon on the cluster to shorten pipeline runtimes.

Worth evaluating, and it reduces time at a higher rate rather than changing the rate class.

D Add spot instances to the worker pool to lower the instance cost.

Touches the cloud infrastructure cost, not the DBU rate class, which is the larger factor here.

ON THE EXAM

The details about autoscaling and sizing are there to close off the tuning answers. The compute type is what is wrong.

> An analytics team of six queries intensively between 9 and 11 each morning, then barely touches the platform. They currently use a classic SQL warehouse, which they leave running all day because it takes several minutes to start.
>
> A colleague objects that serverless has a higher DBU rate and would cost more.

How should the objection be answered?

A The colleague is right; classic is always cheaper per DBU, so it wins.

B Serverless starts in seconds and scales down earlier, so a bursty pattern avoids six hours of idle time despite the higher rate.

C Set a shorter auto-stop on the classic warehouse; the start-up delay is acceptable.

D Move the workload to job compute, which is cheapest of all.

◆ PAUSE AND DECIDE

How should the objection be answered?

A. The colleague is right; classic is always cheaper per DBU, so it wins.
The rate is only half the calculation. You pay rate multiplied by time.

B. Serverless starts in seconds and scales down earlier, so a bursty pattern avoids six hours of idle time despite the higher rate.
The documented argument. Instant availability plus idle termination are only achievable together on serverless.

C. Set a shorter auto-stop on the classic warehouse; the start-up delay is acceptable.
The team already rejected that trade, which is why the warehouse runs all day.

D. Move the workload to job compute, which is cheapest of all.
Job compute is for non-interactive work. This is interactive SQL.

ON THE EXAM

The colleague's fact is correct and their conclusion is not. That is the shape of the reasoning being tested.

> A platform team is asked to attribute Databricks spend to five business units, going back twelve months. They discover that no custom tags were ever configured on any workspace, cluster or warehouse.
>
> They ask whether tags can be applied and backdated across the historical usage.

What should they be told?

A Yes. Applying tags updates the billable usage system table retrospectively.

B No. Tags affect future usage only and cannot be added to past events. Start tagging now, and attribute history another way.

C Yes, but only within the 365-day retention window of the usage table.

D No, but downloading billable usage through the account API produces tagged history.

◆ PAUSE AND DECIDE

What should they be told?

A Yes. Applying tags updates the billable usage system table retrospectively.

Tags are recorded with usage as it happens. Past records are not rewritten.

B No. Tags affect future usage only and cannot be added to past events. Start tagging now, and attribute history another way.

Stated directly in the guidance, and the reason it advises starting detailed from the beginning.

C Yes, but only within the 365-day retention window of the usage table.

Retention governs how long records are kept, not whether they can be edited.

D No, but downloading billable usage through the account API produces tagged history.

The API returns the same records. It cannot add tags that were never captured.

ON THE EXAM

History can still be partly attributed by joining usage to cluster and job identifiers. It is just far more work than a tag would have been.

> A team migrates several batch jobs from classic job compute to serverless. Afterwards the Databricks bill shows a marked rise in DBU consumption, and the platform team raises a concern that the migration was a mistake.
>
> Nobody has looked at the cloud provider invoice.

What should be checked before drawing a conclusion?

A Nothing. A rise in DBUs is a rise in cost.

B The cloud provider invoice, since serverless DBU rates already include the VM cost that was previously billed separately.

C Whether Photon was disabled during the migration.

D Whether the jobs are now running more frequently than before.

◆ PAUSE AND DECIDE

What should be checked before drawing a conclusion?

A Nothing. A rise in DBUs is a rise in cost.

DBUs are one component. For serverless they carry a cost that used to appear elsewhere.

B The cloud provider invoice, since serverless DBU rates already include the VM cost that was previously billed separately.

Classic bills DBUs plus VMs. Serverless folds the VM into the DBU, so DBUs up and infrastructure down can be a wash.

C Whether Photon was disabled during the migration.

Worth knowing generally, and it would not explain a shift of cost between two invoices.

D Whether the jobs are now running more frequently than before.

A reasonable check, and the sentence about the unread cloud invoice is the stronger clue.

ON THE EXAM

Total cost is DBUs plus infrastructure for classic, and DBUs alone for serverless. Comparing one invoice against itself is the error.

> A Structured Streaming job ingests events continuously. The business confirms that dashboards built on the output are reviewed twice a day, and freshness of a few hours is entirely acceptable.
>
> The team currently runs the stream continuously on an autoscaling cluster, and costs are high.

What is the strongest change?

A Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

B Increase the autoscaling minimum so the cluster scales more smoothly.

C Move the stream to spot instances including the driver.

D Keep the continuous stream but enable Photon to shorten each micro-batch.

◆ PAUSE AND DECIDE

What is the strongest change?

A Switch to the availableNow trigger on a schedule, so compute runs only while there is work.

The documented recommendation for incremental workloads without low-latency requirements. Same code, same checkpoint.

B Increase the autoscaling minimum so the cluster scales more smoothly.

Raises the floor on a cluster that should not be running most of the time.

C Move the stream to spot instances including the driver.

The driver should always be on-demand. Losing it ends the job.

D Keep the continuous stream but enable Photon to shorten each micro-batch.

Faster batches on a cluster running 24 hours a day still means a cluster running 24 hours a day.

ON THE EXAM

The business requirement is the lever, not the technology. A continuous source does not compel continuous compute.

> A company shares several large tables with partners in another cloud region using Delta Sharing. Their Databricks bill is stable, but their cloud provider invoice has risen substantially in the same period.
>
> Nothing about their own workloads has changed.

What is the most likely cause?

A Sharing replicates the data to the recipient's region, duplicating storage.

B Data egress charges from the cloud vendor, incurred by the provider when recipients read across regions.

C The recipients' queries run on the provider's compute and appear as DBUs.

D Shared tables are excluded from predictive optimization, so storage grows unchecked.

◆ PAUSE AND DECIDE

What is the most likely cause?

A Sharing replicates the data to the recipient's region, duplicating storage.

Sharing does not replicate. That is its central property.

B Data egress charges from the cloud vendor, incurred by the provider when recipients read across regions.

Documented: sharing avoids replication, but the cloud vendor may charge egress across clouds or regions, and it lands on the provider.

C The recipients' queries run on the provider's compute and appear as DBUs.

Recipients use their own compute, and the Databricks bill is stated to be stable.

D Shared tables are excluded from predictive optimization, so storage grows unchecked.

Invented. Sharing does not change maintenance eligibility.

ON THE EXAM

The split between the two invoices is the clue. A cost on the cloud bill and not the Databricks bill points at network or storage, not compute.